

Department of Computer Engineering

University of Peradeniya



CO1020 – Computer System Programming

Assembly Project – Mini Command-Line Shell

Group 58

E/22/154 – Inuwara S.A.T

E/23/167 – Karunarathna A.V.P.J.P

Introduction

Assembly language is a low-level programming language that provides direct control over a computer's hardware. Unlike high-level languages, assembly interacts closely with the processor's instruction set, making it highly efficient and deterministic but also more complex and error-prone. In this project, we worked with ARM32 assembly, which is designed for ARM processors and widely used in embedded systems. Writing a shell in assembly requires precise management of memory, system calls and register usage giving us deep insight into how operating systems and input/output handling work at the hardware level.

This report presents a comprehensive overview of our ARM32 shell project. It covers the design and implementation of the shell including the core loop, basic command handling and custom features we added. We detail the logic behind each component, explain design decisions and describe how we used stack operations, branching, and memory management effectively. The report also reflects on the development process and team collaboration. Our aim is to demonstrate both our understanding of low-level programming and the working functionality of the shell.

Project Tasks

Task 01 : Base Shell Architecture

The primary goal of task 1 was to construct a minimal interactive shell using ARM32 assembly language. This requires setting up the fundamental components of a working shell interface including memory section, user input handling and a repeating shell loop.

To begin with the **.data** , **.bss** and **.text** sections were properly defined. The **.data** section holds constant strings such as the shell prompt ("shell>"), command keywords and response messages. The **.bss** section declares a 128-bytes uninitialized buffer used to store user input from the terminal. The **.text** section contains all executable code, beginning from the main label, which serves as the entry point of the program.

The heart of this task is the **shell_loop** , which performs the following actions in an infinite cycle:

1. Displays the shell prompt using a system call.
2. Reads a line of input from the user into the **buffer**.
3. Removes the trailing newline character to sanitize input.
4. Passes the cleaned input to the **handle_command** function.
5. Returns to the start of the loop after processing the command.

The system calls for reading and writing use Linux syscall number and are invoked with the **swi 0** instruction. Input is read from file descriptor 0(stdin) and output is written to descriptor 1 (stdout).

This task also required modular implementation, so helper functions like **print_string** , **read_input** , **strlen** and **remove_newline** were implemented to handle low-level operations. Each function properly uses the stack to save and restore registers, preserving the execution context.

Overall, task 1 laid the structural foundation of the shell. It provides an interactive loop, managed user input, and prepared the shell for later tasks involving command parsing and execution logic. The successful completion of this task allowed seamless user interaction and served as the control flow backbone of the entire project.

```

shell_loop:
    ldr r0, =prompt
    bl print_string

    ldr r0, =buffer
    mov r2, #128
    bl read_input

    ldr r0, =buffer
    bl remove_newline

    bl handle_command

    b shell_loop

```

shell_loop Function

```

.section .data
prompt:    .asciz "shell> "

```

```

.section .bss
.lcomm buffer, 128

```

```

.section .text

main:
shell_loop:
    ldr r0, =prompt
    bl print_string

```

.data / .bss / .text sections

```

strlen:
    push {r1, r2, lr}
    mov r1, r0
    mov r2, #0
strlen_loop:
    ldrb r3, [r1, r2]
    cmp r3, #0
    beq strlen_done
    add r2, r2, #1
    b strlen_loop
strlen_done:
    mov r0, r2
    pop {r1, r2, lr}
    bx lr

```

Helper functions

```

read_input:
    push {r1, lr}
    mov r1, r0
    mov r0, #0
    mov r7, #3
    swi 0
    pop {r1, lr}
    bx lr

```

```

print_string:
    push {r1, r2, lr}
    mov r1, r0
    bl strlen
    mov r2, r0
    mov r0, #1
    mov r7, #4
    swi 0
    pop {r1, r2, lr}
    bx lr

```

```

remove_newline:
    push {r1, r2, lr}
    mov r1, r0
remove_loop:
    ldrb r2, [r1]
    cmp r2, #0
    beq remove_done
    cmp r2, #10
    beq set_zero
    add r1, r1, #1
    b remove_loop
set_zero:
    mov r2, #0
    strb r2, [r1]
remove_done:
    pop {r1, r2, lr}
    bx lr

```

Helper functions

- `print_string`
Displays message such as the shell prompt(`shell>`), command outputs, or errors by using the Linux `write` syscall.
- `read_input`
Read the user's command input each time the shell loop runs, using the Linux `read` syscall
- `strlen`
Calculate the length of a null-terminated string.
- `remove_newline`
Scans and removes the newline character(`\n`) from the user's input buffer.

Task 2 - Command Handling Logic

In task 2, we implemented the code for recognizing and executing user commands. This involved comparing the user's input with predefined command string and invoking the appropriate functions for each command. The focus was on making the shell interactive, modular and extensible.

The command input from the user is first read into a buffer and then sanitized using the **`remove_newline`** function. Afterward, the **`handle_command`** function is responsible for determining which command was entered. It uses a custom **`strcmp`** function to compare the input with predefined command keyword such as **`"hello"`**, **`"help"`**, **`"exit"`** and **`"clear"`**. Each matching condition uses conditional branching(`cmp`, `beq`, `bne`) to determine the flow of execution.

To maintain modularity and readability, each command is handled in separate function.

- `hello_command` prints a "Hello World!" message.

```
hello_command:
    push {lr}
    ldr r0, =hello_msg
    bl print_string
    pop {lr}
    bx lr
```

- `help_command` displays a list of all available commands.

```
help_command:
    push {lr}
    ldr r0, =help_msg
    bl print_string
    pop {lr}
    bx lr
```

- `exit_command` terminated the shell.

```
exit_command:
    mov r7, #1
    mov r0, #0
    swi 0
```

- `clear_command` sends ANSI escape codes to clear the terminal screen.

```
clear_command:
    push {lr}
    ldr r0, =clear_seq
    bl print_string
    pop {lr}
    bx lr
```

Additionally the `handle_command` function ensures that an unknown command is caught and give output sing the “Unknown command.\n” message.

Each functions in this task uses proper register saving and resorting with the stack(push and pop instructions) to preserve execution context and avoid register corruption.

By successfully completing task 2, we established the foundation of a functional CLI interface with multiple commands.

Helper functions

- `handle_command`

This function is responsible for comparing the user’s input against known commands and routing control to the appropriate handler function.

It preserve link register using `push {lr}` and `pop {lr}` to safely return after command handling.

```

handle_command:
    push {lr}

    ldr r0, =buffer
    ldr r1, =cmd_hello
    bl strcmp
    cmp r0, #0
    beq call_hello

    ldr r0, =buffer
    ldr r1, =cmd_help
    bl strcmp
    cmp r0, #0
    beq call_help

    ldr r0, =buffer
    ldr r1, =cmd_exit
    bl strcmp
    cmp r0, #0
    beq call_exit

    ldr r0, =buffer
    ldr r1, =cmd_clear
    bl strcmp
    cmp r0, #0
    beq call_clear

    ldr r0, =buffer
    ldr r1, =cmd_hex
    bl starts_with
    cmp r0, #0
    beq call_hex

    ldr r0, =buffer
    ldr r1, =cmd_sum
    bl starts_with
    cmp r0, #0
    beq call_sum

    ldr r0, =unknown_msg
    bl print_string
    b handle_done

```

```

cmd_hello: .asciz "hello"
cmd_help: .asciz "help"
cmd_exit: .asciz "exit"
cmd_clear: .asciz "clear"
cmd_hex: .asciz "hex "
cmd_sum: .asciz "sum "

hello_msg: .asciz "Hello World!\n"
help_msg: .asciz "Available commands:\n - hello\n - h
unknown_msg: .asciz "Unknown command.\n"
clear_seq: .asciz "\033[2J\033[H"
hex_prefix: .asciz "0x"
newline: .asciz "\n"

```

Example Inputs and Outputs

```
tharushainuwara@MSI:/mnt/c/Users/Tharusha/Desktop/ASM project$ qemu-arm -L /usr/arm-linux-gnueabi ex
shell> help
Available commands:
- hello
- help
- exit
- clear
- hex <num>
- sum <num1> <num2>
shell> hello
Hello World!
shell> exit
tharushainuwara@MSI:/mnt/c/Users/Tharusha/Desktop/ASM project$
```

Before and after give clean command:

Before

```
tharushainuwara@MSI:/mnt/c/Users/Tharusha/Desktop/ASM project$ qemu-arm -L /usr/arm-linux-gnueabi ex
shell> hello
Hello World!
shell> help
Available commands:
- hello
- help
- exit
- clear
- hex <num>
- sum <num1> <num2>
shell> hi
Unknown command.
shell> clear
```

After

```
shell>
```


Task 3 : Custom Commands

In this task, we implemented two custom commands. There are hex and sum.

1. hex <number>

This command converts a user supplied positive integer into its hexadecimal representation and display it with a 0x prefix.

Input format

```
shell> hex 255
```

Implementation method

- **handle_command** uses **starts_with** to detect the "hex " prefix.
- The command string following "hex " is passed to **hex_command**.
- **Hex_command**:
 - Parses the input string character by character.
 - Multiplies the accumulated value by 10 and adds each digit (ASCII to int).
 - Calls **print_string** to show "0x", then **print_hex** to display the number in base 16.
- **Print_hex** uses recursion to print digits from the most significant to least significant, converting values 0–15 into ASCII digits or hex letters (a–f).

Output

```
shell> hex 255
0xff
shell> 
```

2. sum <num1> <num2>

This command calculate and prints the sum of two integers and output only positive answers.

Input format

```
shell> sum 100 147
```

Implementation method

- Recognized using **starts_with** with the "sum " prefix.
- Passes the remainder of the input string to **sum_command**.

- **sum_command:**
 - Skips leading spaces using **skip_spaces**.
 - Parses the first number with **parse_number**.
 - Skips to the second number and parses it too.
 - Adds the two numbers in add r4, r2, r3.
 - Prints the result using **print_decimal**.
- Uses the **__aeabi_uidivmod** library call for printing each digit of a decimal number via recursive division.

Output

```
shell> sum 100 147
247
shell> █
```

External ARM directives

1. .extern __aeabi_uidivmod

This declares an external function provided by the ARM runtime that performs unsigned integer division and modulo operations. We use it because many ARM processors don't have built-in hardware division instructions, so this software routine handles division and remainder calculations efficiently when this assembly code needs to perform these operations.

2. clear_seq: .asciz "\033[2J\033[H"

This defines an ANSI escape sequence string used to clear the terminal screen and move the cursor to the home position (top-left corner). Here, \033 is the escape character, [2J clears the entire screen, and [H moves the cursor to the home position. This sequence is used in the shell's clear command to reset the terminal display.

3. push and pop

The push and pop instructions in ARM assembly are used to manage the stack by saving and restoring register values across function calls. We use push at the beginning of a function to store important registers—such as r4, which may hold temporary data, and lr (link register), which stores the return address—so that their original values are not lost when the function modifies them. Later, pop is used at the end of the function to retrieve and restore those values. This ensures the program returns to the correct

location and continues with the correct register contents. Using push and pop is essential for maintaining program stability, especially when dealing with nested functions or interrupts.

4. bx lr

The bx lr instruction in ARM assembly is used to return from a function. The lr (link register) holds the return address, which is automatically set when a function is called using the bl (branch with link) instruction. By executing bx lr, the processor jumps back to the address stored in lr, effectively returning control to the calling function. This is essential for proper function call and return behavior in ARM assembly. The bx instruction also supports switching between ARM and Thumb instruction sets if needed, making bx lr both a return and a mode-aware jump instruction.

5. swi

The swi (Software Interrupt) instruction in ARM assembly is used to invoke system calls provided by the operating system. When swi is executed, the processor switches to supervisor mode and jumps to a predefined exception handler, allowing your program to request OS-level services such as reading input, writing to the screen, or exiting. For example, swi 0 is commonly used with specific values in registers (like r7 for the syscall number) to perform actions such as printing text (write) or reading input (read).

Individual contribution

Our project was a collaboration effort where both members actively participated throughout all phases. Initially, we met to discuss the overall shell design and planned how to approach each task systematically. I took responsibility for Task 1, focusing on implementing the main shell loop, reading user input, and basic command processing. Meanwhile, my partner developed Task 2, which involved extending command handling and adding logic for specific built-in commands.

After completing these individual tasks, we joined forces to work on Task 3, where we designed and implemented additional commands like hex and sum. During this phase, we collaborated closely to debug the code, optimize functionality, and ensure smooth integration of all components. Both of us also shared responsibilities in documenting the project report and preparing the final demonstration. Our teamwork and regular communication helped us resolve challenges effectively and achieve the project goals on time.

AI Tool Usage

We used AI tools such as ChatGPT during the project mainly for learning and clarification. These tools were especially helpful when we encountered errors in our assembly code and needed help understanding what caused them. We also used them to learn about unfamiliar ARM assembly directives and instructions (such as swi, bx lr, push, pop), and how to use them correctly in our program. However, the final implementation and design decisions were made entirely by us. The AI was only used as a support tool to enhance our understanding and improve our learning process.

How compile and run

Compile

```
tharushainuwara@MSI:/mnt/c/Users/Tharusha/Desktop/ASM project$ arm-linux-gnueabi-gcc -Wall shell.s -o shell
```

Run

```
tharushainuwara@MSI:/mnt/c/Users/Tharusha/Desktop/ASM project$ qemu-arm -L /usr/arm-linux-gnueabi shell
```

```
shell> help
Available commands:
- hello
- help
- exit
- clear
- hex <num>
- sum <num1> <num2>
shell> █
```